

SKILL-3

Roll No. : 2520030437

Name : Bandla Venkata Rama Vinay

SEC-8A

SKILL-3:

Aim: Develop a simple command-history program in C that accepts commands from the user and maintains a history of previously entered commands using dynamically allocated memory.

Implement commands to display the stored history, retrieve the previous command, report the absence of a newer command, and terminate the program while releasing the memory allocated for command history.

Procedure :

1. Define a maximum command history size of **10** using **MAX_HISTORY**.
2. Read commands from the user using **fgets()** and remove the trailing newline.
3. Store non-empty commands using **malloc()** and **strcpy()**.
4. Use **history** to display all stored commands.
5. Use **prev** to display the most recently stored command and **next** to report that there is no newer command.
6. Use **exit** to terminate the program and release all stored history using **free()**.

Output :

```
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ ./Skill_3
Simple Command History
Type 'history' to display commands
Type 'prev' for previous command
Type 'next' for next command
Type 'exit' to quit

shell> history
shell> prev
No previous command.
shell> next
Next command: No newer command.
shell> exit
History memory released.
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ |
```

The submitted execution shows the Simple Command History interface. The commands **history**, **prev**, **next**, and **exit** are recognized. With no previous command available, the program reports **No previous command.**; the **next** command reports **No newer command.**. On exit, the program reports **History memory released.**

Code:

```
Skill > c Skill-3.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAX_HISTORY 10
6
7  int main()
8  {
9      char *history[MAX_HISTORY];
10     int count = 0;
11     char input[100];
12
13     printf("Simple Command History\n");
14     printf("Type 'history' to display commands\n");
15     printf("Type 'prev' for previous command\n");
16     printf("Type 'next' for next command\n");
17     printf("Type 'exit' to quit\n\n");
18
19     while (1)
20     {
21         printf("shell> ");
22         fgets(input, sizeof(input), stdin);
23         input[strcspn(input, "\n")] = '\0';
24
25         if (strcmp(input, "exit") == 0)
26             break;
27
28         if (strcmp(input, "history") == 0)
29         {
30             for (int i = 0; i < count; i++)
31                 printf("%d %s\n", i + 1, history[i]);
32             continue;
33         }
34
35         if (strcmp(input, "prev") == 0)
36         {
37             if (count > 0)
```

Code (continued):

```
38         printf("Previous command: %s\n", history[count - 1]);
39     else
40         printf("No previous command.\n");
41     continue;
42 }
43
44 if (strcmp(input, "next") == 0)
45 {
46     printf("Next command: No newer command.\n");
47     continue;
48 }
49
50 if (strlen(input) > 0)
51 {
52     if (count < MAX_HISTORY)
53     {
54         history[count] = malloc(strlen(input) + 1);
55         strcpy(history[count], input);
56         count++;
57     }
58
59     printf("Command stored in history.\n");
60 }
61 }
62
63 for (int i = 0; i < count; i++)
64     free(history[i]);
65
66 printf("History memory released.\n");
67
68 return 0;
69 }
```

Functions and Program Operations

Function / Operation	Purpose
fgets()	Reads a command from standard input.
strcmp()	Compares the entered command with history, prev, next, and exit.
strcspn()	Locates the newline so it can be removed from the input.
strlen()	Determines the required memory size for a command.
malloc()	Dynamically allocates memory for a stored command.
strcpy()	Copies the entered command into allocated history memory.
free()	Releases the dynamically allocated history memory.

History Management

The program maintains an array of up to **10** command pointers. Whenever a non-empty command is entered, memory is allocated according to its length, the command is copied into that memory, and **count** is incremented.

The **history** command iterates from index 0 to **count - 1** and displays each stored command with its corresponding number.

Previous and Next Commands

The **prev** operation checks whether at least one command exists. If so, it prints the most recently stored command using **history[count - 1]**. The **next** operation in this implementation reports that there is no newer command.

Observation :

- The program starts by displaying instructions for the supported commands.
- The command buffer has a size of 100 characters.
- Up to 10 commands can be stored in the history.
- Commands are dynamically allocated using **malloc()** before being stored.
- The **history** command displays stored commands with their numbers.
- The **prev** command displays the most recently stored command when history is available.
- The **next** command reports that there is no newer command.
- Before termination, the program releases every allocated history entry using **free()**.

Result :

The C program successfully implements a simple command-history mechanism with support for **history**, **prev**, **next**, and **exit**. Commands are stored using dynamic memory allocation, and the allocated memory is released before program termination. The submitted execution completed successfully and displayed the expected command responses.

SKILL-3 Completed Successfully